Model Context Protocol (MCP) — Complete Guide

Model Context Protocol (MCP) is an open standard developed by Anthropic in 2024 that defines how AI assistants communicate with external tools, data sources, and services. MCP solves the integration problem — before MCP, every AI application had to build custom connectors for every tool, leading to duplicated effort and inconsistent behavior.

1. What is MCP?

MCP stands for Model Context Protocol. It is a standardized communication protocol that allows large language models (LLMs) to interact with external systems in a consistent, secure, and predictable way. Think of MCP as a universal plug standard — just like USB allows any device to connect to any computer, MCP allows any AI model to connect to any tool or data source that implements the protocol.
Before MCP existed, AI developers had to write custom code every time they wanted to connect an LLM to a new tool. This meant duplicated work, security inconsistencies, and fragile integrations. MCP eliminates this problem by providing one standard that all tools and models can speak.

2. Core Architecture of MCP

MCP follows a client-server architecture with three main components:

MCP Host: The application that runs the AI model. Examples include Claude Desktop, an IDE plugin, or a custom AI application. The host is responsible for managing connections to MCP servers and routing requests.

MCP Client: A component inside the host that speaks the MCP protocol. Each client maintains a one-to-one connection with a single MCP server. The client handles the low-level protocol details so the host does not have to.

MCP Server: A lightweight program that exposes specific capabilities — tools, resources, or prompts — to the AI model. A server could expose a file system, a database, a web search API, or any other external capability. Servers can be local (running on the user's machine) or remote (running in the cloud).

The communication between client and server uses JSON-RPC 2.0 as the message format. JSON-RPC is a simple remote procedure call protocol encoded in JSON, which makes it easy to implement in any programming language.

3. The Three Primitives of MCP

MCP servers can expose three types of capabilities, called primitives:

Tools: Functions that the AI model can call to perform actions or retrieve information. Examples include searching the web, running a database query, sending an email, or reading a file. Tools are the most commonly used primitive. Each tool has a name, a description (which the LLM reads to decide when to use it), and an input schema defining what parameters it accepts.

Resources: Static or dynamic data that the AI model can read. Resources are identified by URIs (Uniform Resource Identifiers). Examples include a file at file:///home/user/document.txt, a database record at db://customers/123, or a live feed at api://weather/dallas. Unlike tools, resources do not perform actions — they only provide data.

Prompts: Reusable prompt templates that servers can provide. These are pre-written instructions or conversation starters that the AI can use. For example, a coding server might provide a prompt template called 'review_code' that automatically formats a code review request properly.

4. How MCP Works — Step by Step

When an AI application uses MCP, the following sequence of events occurs:

Step 1 — Initialization: The MCP host starts and connects to one or more MCP servers. The client sends an initialize request containing the protocol version and client capabilities.

Step 2 — Capability Discovery: The server responds with its capabilities — the list of tools, resources, and prompts it offers. The client stores this information.

Step 3 — User Request: The user asks the AI a question or gives it a task. The AI model receives the user message along with the list of available tools from all connected servers.

Step 4 — Tool Selection: The LLM reads the tool descriptions and decides which tool (if any) to call. This decision is based entirely on the tool's name and description — this is why writing clear, accurate tool descriptions is critical.

Step 5 — Tool Execution: The client sends a tools/call request to the appropriate server. The server executes the tool and returns the result.

Step 6 — Response Generation: The LLM receives the tool result and uses it to generate a final response to the user.

5. Transport Mechanisms

MCP supports two transport mechanisms for communication between clients and servers:

stdio (Standard Input/Output): The client launches the server as a subprocess and communicates through stdin and stdout pipes. This is the simplest transport and is used for local servers running on the same machine. It is the most common transport for desktop applications like Claude Desktop.

HTTP with SSE (Server-Sent Events): The client connects to a remote server over HTTP. The server uses Server-Sent Events to push messages back to the client. This transport is used for remote servers hosted in the cloud and enables sharing MCP servers across multiple users.

6. Security in MCP

Security is a core design principle of MCP. The protocol includes several security features:

Explicit User Consent: Before an AI can use any tool, the user must explicitly approve it. MCP hosts are required to show users what tools are available and get consent before enabling them. This prevents AI models from secretly accessing sensitive data or performing actions the user did not authorize.

Sandboxing: Each MCP server runs in isolation. A server handling file access cannot interfere with a server handling email. This limits the blast radius of any security issue.

Minimal Permissions: MCP encourages the principle of least privilege — servers should only request the permissions they actually need. A weather server does not need file system access.

Local Execution: For sensitive data, servers can run entirely on the user's local machine, ensuring that private information never leaves the user's computer.

7. MCP vs Traditional API Integration

Before MCP, connecting an AI to external tools required custom API integration for every tool. This had several problems: N x M complexity (N models times M tools equals N times M custom integrations), inconsistent behavior across tools, security handled differently everywhere, and no standard for capability discovery.

With MCP, the complexity becomes N + M. Each model implements MCP once, each tool implements MCP once, and they all work together automatically. This is the same insight that

made USB so successful in hardware.

8. Popular MCP Servers

The MCP ecosystem has grown rapidly since its introduction. Some widely used MCP servers include:

Filesystem Server: Provides tools for reading, writing, and managing files on the local file system. Supports operations like read_file, write_file, list_directory, and create_directory.

GitHub Server: Connects to the GitHub API, allowing AI models to read repositories, create issues, submit pull requests, and manage code. Useful for AI-powered development workflows.

PostgreSQL Server: Allows AI models to query PostgreSQL databases using natural language. The AI converts natural language to SQL, executes it, and returns results.

Brave Search Server: Provides web search capabilities through the Brave Search API. Allows AI models to search the web for current information.

Slack Server: Integrates with Slack workspaces, allowing AI models to read channels, send messages, and manage workspace data.

9. Building an MCP Server

Building an MCP server requires implementing the MCP protocol in your chosen language. Anthropic provides official SDKs for Python and TypeScript. A minimal Python MCP server looks like this:

First, you install the MCP SDK using pip. Then you create a server instance and define tools using decorator syntax. Each tool is a Python function with a clear docstring that describes what it does. The server handles all the protocol details automatically — you only need to write the business logic of your tools.

The Python SDK uses the @server.tool() decorator to register functions as MCP tools. The function name becomes the tool name, and the docstring becomes the description that the LLM reads. Input parameters are defined using Python type hints, which the SDK automatically converts to a JSON schema.

10. MCP in the AI Ecosystem

MCP was released by Anthropic as an open standard in November 2024. Since then, it has been adopted by major players in the AI industry. OpenAI announced support for MCP in their products in early 2025. Google has also announced MCP compatibility for Gemini models.

The MCP specification is maintained publicly on GitHub and accepts community contributions. The protocol is versioned, with the current version being 2024-11-05. Backward compatibility is a key design goal — older servers continue to work with newer clients.

As of 2025, there are over 1000 publicly available MCP servers covering everything from productivity tools and databases to IoT devices and scientific instruments. The protocol has become the de facto standard for AI tool integration, fulfilling its original goal of being the USB of AI.

Summary

MCP is an open protocol by Anthropic that standardizes how AI models connect to external tools and data. It uses a client-server architecture with three primitives: tools, resources, and prompts. Communication uses JSON-RPC 2.0 over stdio or HTTP+SSE transports. Security is enforced through explicit user consent and sandboxing. MCP reduces integration complexity from N x M to N + M, making it easy for any AI to use any tool.